Chapter 7
Enhancing FPGA CAD Flow with AI-Powered Solutions



Chunxiao Lin and Yang Yi

7.1 What Is FPGA?

Field-Programmable Gate Arrays (FPGAs) are a category of programmable logic devices that use electronic components to build reconfigurable digital circuits. Among these, Read-Only Memories (ROMs) and Mask Programmable Gate Arrays (MPGAs) belong to factory-programmable devices. They are programmed during the manufacturing process. This programming is permanent and aims at high-volume and application-specific design. Another type of device, called field-programmable devices, allows users to reprogram the devices after fabrication. Simple programmable logic devices (SPLDs), complex programmable logic devices (CPLDs), and FPGAs all belong to this type. SPLDs are suitable for the implementation of small-scale logic circuits, with limited flexibility and complexity. CPLDs are made up of several SPLDs with programmable switch matrices in between. This provides better flexibility, which is, however, still restricted by the architecture of sea of gates. The development of FPGAs solved the problem by adopting the RAM-based lookup table (LUT) structure. Compared to all other programmable devices, FPGAs can achieve great flexibility and support for large-scale circuit design.

Figure 7.1 shows the basic architecture of an FPGA. There are three main elements: configurable logic blocks (CLBs), switch matrices, and programmable I/O blocks. CLBs are the primary building blocks of FPGAs, including LUTs and flip-flops (FFs) for the implementation of combinational logic and sequential logic. Switch matrices are used to route signals between different CLBs and other components within the FPGA. This supports the implementation of complex circuits. The interaction between the FPGA and external components, such as

C. Lin (✉) · Y. Yi
Bradley Department of Electrical and Computer Engineering, Virginia Tech, Blacksburg, VA, USA
e-mail: chunxiaol@vt.edu; yangyi8@vt.edu

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2025

A. Iranmanesh, H. Sayadi (eds.), *AI-Enabled Electronic Circuit and System Design*,
https://doi.org/10.1007/978-3-031-71436-8_7

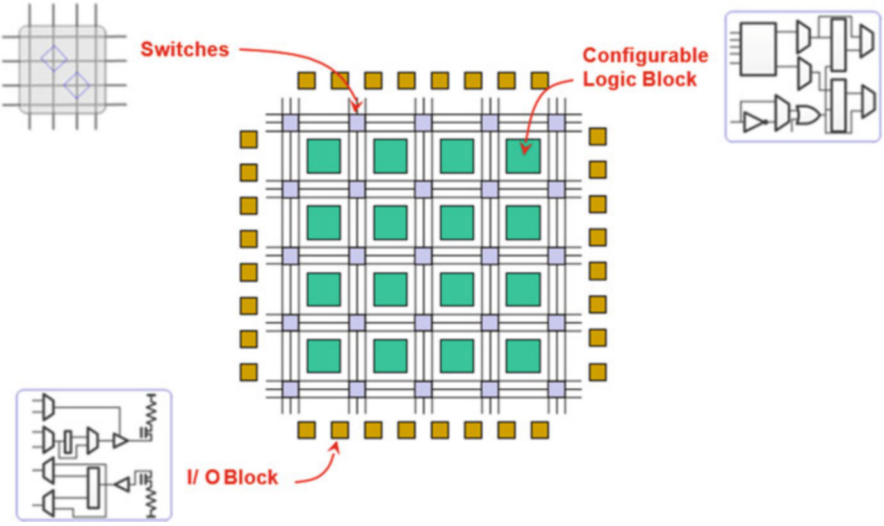


Fig. 7.1 The FPGA structure with I/O block, logic block, and switches

the processor, is enabled by programmable I/O blocks. Moreover, there are more specialized programmable function blocks like ALUs (arithmetic logic units), block RAMs, and DSP (digital signal processing) slices, which are distributed throughout the board and not displayed in Fig. 7.1. Therefore, all these components enable the ultrahigh programmability of FPGAs.

FPGA, as a hardware development platform, also shows its irreplaceability in comparison with other platforms. Unlike CPUs with powerful scheduling capability, full programmability but low computational complexity and high power consumption, GPUs with high parallelism but only within a single application, and ASICs with ultralow cost, low power consumption, but time-consuming fabrication, FPGAs achieve a balance between the programmability and the power consumption, with both pipeline parallelism and data parallelism for higher concurrent processing, and a relatively shorter development cycle.

Due to their ability to create complex digital circuits directly in hardware, FPGAs are widely used in several fields. This includes data centers [1, 2], the automobile [3, 4], telecommunications [5–8], aerospace [9–11], and defense industries [12–14]. The capacity for high-speed parallel processing has also helped FPGAs become better known in recent years in the fields of artificial intelligence and machine learning [15–19].

Currently, the top FPGA manufacturers are listed below, with more details listed in Table 7.1.

- **Xilinx** currently holds the dominant position in the global market for FPGAs, with a market share exceeding 50%. The target market of Xilinx includes Communications, Industrial, Data Center, and Aerospace and Defence markets.

Table 7.1 The latest FPGA products from the top companies

Company	FPGA products	Main markets	Design tools
Xilinx	UltraScale+, UltraScale, Virtex-7	Communications, aerospace and defence, data center	Vivado
Intel (Altera)	Stratix, Arria, MAX, Cyclone	Communications, military, automotive	Quartus
Lattice	Avant, CertusPro, CrossLink, MachXO5	Communications, computing, industrial, consumer	lattice diamond
Microchip	RTG4, SmartFusion2	Aerospace and defence, medical, industrial	Libero
Achronix	Speedster7t, Speedcore	High-performance computing, networking and telecom	ACE
QuickLogic	EOS S3, ArcticLink3, PolarPro3	Displays, audio and voice, mobile and IoT	Sensor development kit
Flex logic	EFLX	Embedded apps	Inference compiler

The company provides a wide range of FPGA options, including Versal ACAP (Adaptive Compute Acceleration Platform), UltraScale+, UltraScale, 7-series, Spartan-7, and previous generations, making it the most extensive portfolio in the industry. For the latest Versal ACAP portfolio, which utilizes the TSMC 7-nm FinFET process technology, is the first platform to combine software programmability and domain-specific hardware acceleration. The portfolio provides six device series, each uniquely designed to offer scalability and AI inference capabilities for a wide range of applications, from cloud and networking to wireless communications and edge computing.

- **Intel** acquired Altera in 2015 and now offers FPGAs under the Intel brand. As the second largest FPGA supplier with around 30% market share, Intel shows a strong presence in the communications infrastructure and military/aerospace markets.

  This company also provides a wide variety of FPGA series, including Intel Agilex, Stratix, Arria, MAX, and Cyclone. The Intel Agilex FPGA Portfolio, for example, is built on Intel 10 SuperFin and Intel 7 technology, which greatly enables customized acceleration.
- **Lattice Semiconductor** is another influential FPGA company, focusing on the low-power FPGA market, where the problems are across the network, from the Edge to the Cloud, mainly in communications, computing, industrial, automotive, and consumer markets.

  In Lattice's product lines, there are Avant, CertusPro-NX as general-purpose FPGAs, CrossLink-NX for video connection FPGAs, MachXO5-NX for control and security FPGAs, and iCE40 UltraPlus, iCE40 Ultra/UltraLite for ultralow power FPGAs. With an ultra-low power advanced process with static current as low as 75 μA and 1–10 mA active current, iCE40 UltraPlus FPGA can achieve low power connectivity and computing.

- In addition to the companies mentioned above, there are other important players in the FPGA market. **Microchip (formerly Microsemi)** is a leading supplier of FPGAs for the defense and aerospace industries. The company's FPGAs for space applications are manufactured and tested using specialized processes to ensure the highest standards of quality and reliability for satellite payloads and launch vehicles. **Achronix Semiconductor** offers the Speedster7t FPGA family, which is optimized for high-bandwidth workloads and eliminates performance bottlenecks associated with traditional FPGAs. Speedster7t FPGAs feature a revolutionary new 2D network-on-chip (2D NOC), which can deliver ASIC-level performance while retaining the full programmability of FPGAs. **QuickLogic** develops low-power FPGAs for the portable electronics market. Their products, the EOS S3, ArcticLink, and PolarPro 3 FPGAs, target energy-efficient and battery-powered devices.

While FPGAs offer significant advantages in terms of customization, low latency, and power efficiency, as shown above, they also come with certain limitations. One of the main challenges is the complexity of design and programming. Unlike CPUs and GPUs, which have mature and user-friendly software ecosystems, FPGAs require a deep understanding of hardware design and low-level programming. This potentially can be a big barrier for developers in other fields. Additionally, while FPGAs can be reprogrammed to perform specific tasks efficiently, they generally do not match the raw computational power of GPUs, which show extreme performance in data computation with regular parallelism and high floating-point compute throughput. Therefore, FPGAs are often not the best choices for general-purpose computing that requires high computational throughput. Finally, the cost of FPGA devices can be higher compared to other hardware options, such as ASIC chips. This makes them less accessible for small-scale projects with tight budgets. Despite these challenges, FPGAs continue to be used in a variety of applications where their unique strengths provide a competitive advantage.

To meet computational requirements in the AI era, the integration of AI modules in FPGA has become a significant trend in the current technological landscape. Companies like AMD Xilinx, Intel, and Achronix have introduced FPGA boards that incorporate AI modules, enhancing their performance and computational capabilities. Xilinx's Versal AI core and Alveo AI Acceleration Platforms provide high-performance AI inference and adaptable computing. Intel introduces the Agilex 7 FPGA and FPGA AI Suites with high-bandwidth memory and AI-optimized DSP blocks. Achronix Speedster7t features advanced interconnect for high-speed data transfer, AI/ML optimization, and a large number of logic elements and DSP blocks.

These AI-integrated FPGAs offer autonomous decision-making, real-time adaptation to new data, and intelligent information processing. The AI modules boost FPGA performance by allowing for faster response times and enabling more specialized, innovative operations. With the ability to customize processing functions at the basic logic port level, AI-integrated FPGAs can become better accelerator options than GPUs in specific scenarios.

7.2 Overview of the FPGA CAD Flow

FPGA CAD (Computer-Aided Design) flow is a series of steps that designers need to follow to implement or program any circuit on the FPGA chips. The mainstream FPGA synthesis tools from the companies above are all designed to assist designers based on the steps. Figure 7.2 illustrates the typical FPGA CAD flow, from the initial Design Entry to the final Bitstream Generation.

In Design Entry, designers will conceptualize their ideas about the function and structure of their circuits and transform them into a computerized representation, which is called RTL (Register-Transfer Level) design. RTL is a level of abstraction in digital circuit design where the behavior of the circuit is described in terms of the flow of data between registers. In this process, specialized computer languages called Hardware Description Languages (HDLs) are adopted, such as VHDL, Verilog, and SystemVerilog. As their names imply, HDLs are only tools for circuit description instead of tools for circuit design. When it comes to a large and complex system, HDL programming can be difficult and time-consuming. Therefore, high-level synthesis (HLS) was proposed to automate the process, which allows designers to build and verify their designs more efficiently by coding only in

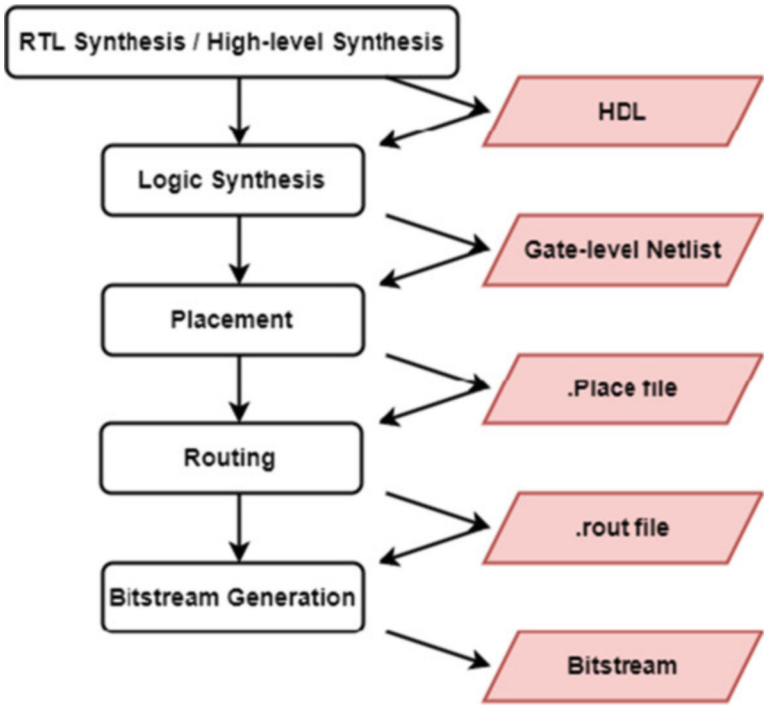


Fig. 7.2 An overview of the FPGA CAD flow

high-level programming languages such as C and C++. By enabling a higher level of abstraction, HLS in FPGA CAD can simplify algorithm expression, accelerate development cycles, and facilitate easier verification.

In Logic Synthesis, the design at the RTL level will be forwarded to the gate level. With the synthesis tool of the selected FPGA vendor and model, designers can feed the HDL files to it and generate the netlist that is built with the primitives provided by the vendor. There are usually additional built-in functions in the synthesis tools, such as logic optimization and timing analysis. These techniques can further improve the performance of the generated netlist.

After generating the synthesized netlist, the next task is to assign all the primitives to discrete slots on the FPGA board, which is called placement. Placement is one of the most time-consuming steps in the whole FPGA CAD flow. Designers need to decide on the physical location of each logic component using synthesis tools to optimize wire length and timing. For example, there are 485,760 logic cells and 37,080 kb memory on the Xilinx Virtex-7 VC 707 FPGA board, which a designer needs to select from. Two main tasks in placement are congestion estimation and routability prediction, focusing on the management of the routing resources. Smart placement will reduce the work load of the subsequent routing task and lead to a final design with better performance.

Routing is the step in the FPGA CAD flow to establish interconnections between the placed logic elements based on the placement results. It ensures timing constraints, minimizes delays, minimizes power consumption, and ensures signal integrity and resource utilization. Traditional methods are based on Dijkstra's shortest path algorithm, with signals to negotiate routing resources. Since the routing resources on an FPGA board are limited, routing is also a time-consuming task in large digital system designs.

In the end, the design tools will compile all the configuration data such as the specific configuration of logic elements including LUTs and flip-flops, the placement and routing details, and I/O configuration into a binary file known as bitstream. Then the bitstream will be sent from the host machine to program the flash memory of the FPGA board.

7.3 AI-Powered HDL Generation

The Mead-Conway approach, initiated in the late 1970s, marked a significant revolution in the field of VLSI (Very Large-Scale Integration) chip design. The revolution brought the possibility for computer engineers and scientists to engage in chip design without the need for extensive knowledge in electrical engineering. Hardware designers can then use HDLs to compile a design into physical chips.

However, the scale of hardware design is increasing at an exponential rate. Better abstraction of programming languages has been explored to reduce workload. One of the design abstraction improvements is the register-transfer level (RTL). It allows

designers to describe the circuit based on the flows of digital signals between registers and the logic performance based on these signals.

High-level synthesis is another abstraction improvement after RTL. Instead of working on individual registers and operations in each clock cycle, HLS operates on a higher level abstraction. Hardware can be described using higher level programming languages, such as C, C++, and SystemC, which further simplifies the design process.

7.3.1 Conventional High-Level Synthesis

A typical HLS process is shown in Fig. 7.3. It begins with the hardware description in high-level programming language. The code is processed by the front-end compilation and then by the back-end hardware generation. In this way, the finite-state machine (FSM) and datapath components are generated, which will be used to produce the low-level HDL code.

According to Cadence's assessment, their HLS product line has the potential to cut down design and verification costs by 25–50%. This has led major companies such as Synopsys, Mentor, Intel, and Xilinx to create their own HLS design automation tools to address power and timing issues, thus increasing efficiency in custom hardware design [20, 21]. In [22], an overview of academic and commercial HLS tools is provided, in which the HLS tools are classified according to the input language. We can differentiate between two primary types: HLS tools that use domain-specific languages (DSLs) and tools that are based on general-purpose programmable languages (GPLs). DSLs can be divided into newly created languages specifically designed for a particular tool flow and dialects of GPLs,

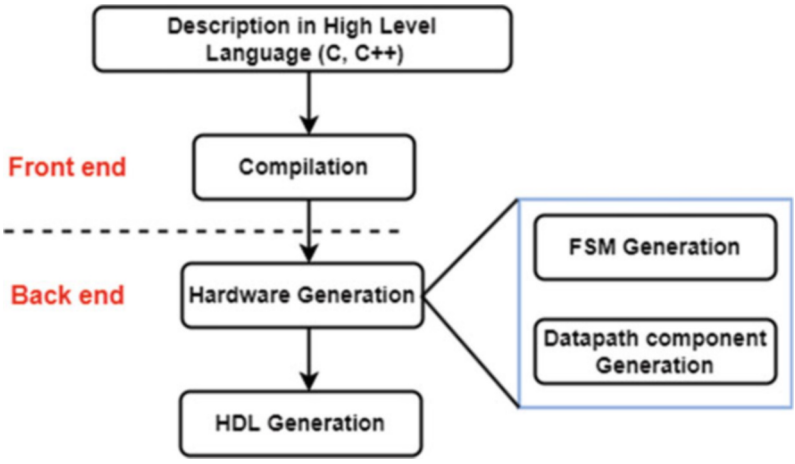


Fig. 7.3 Conventional High-Level Synthesis (HLS) process

which are languages derived from a GPL (e.g., C) and augmented with specific elements to communicate hardware information to the tool.

Although HLS shows great improvements in the development speed of the HDL generation, the generated circuits cannot provide the same quality as the manually generated ones. This is due to the following limitations that impact the hardware quality:

1. The high-level design process of HLS conceals potential optimization opportunities.
2. The design space exploration is timing-consuming, and the current HLS tools are not fully developed for the prediction of all the performance metrics.
3. The standard sets of code optimization in CPU, like LLVM, are not suitable for an FPGA back-end.

Therefore, the top two objectives of AI-based research in HLS are to (1) achieve better accuracy in the prediction of the performance and resource utilization and (2) explore the design space.

7.3.2 ML-Based Metric Prediction Algorithms in HLS

High-level synthesis tools have been powerful enough today after decades of development. For widely used HLS tools, such as the Vitis HLS from Xilinx [23] and Intel HLS [24], they can already reduce design efforts and time to market significantly. In addition to HDL generation, metric estimation is another necessary function for each HLS tool. The key metrics to evaluate the FPGA design are timing, latency, resource utilization, and power consumption. Timing analysis provides insight into the clock cycle performance, ensuring that the design meets the necessary speed requirements. Latency estimation reveals the number of clock cycles required for a particular operation or a set of operations, which is crucial for applications with strict performance constraints. Resource utilization metrics offer a detailed amount of the available resources onboard—such as logic elements, DSP blocks, and memory—that are being used by the design. This information is vital for ensuing that the design fits within the target FPGA and for identifying areas where resources can be optimized. Power consumption is another important metric, especially for designs with requirements of energy efficiency. HLS tools estimate the power usage of the design, allowing designers to make informed decisions to minimize power consumption without compromising performance. By providing detailed metrics on timing, latency, resource utilization, and power consumption, HLS tools empower designers to make data-driven decisions to meet their design requirements efficiently and effectively.

However, in most cases, the HLS-estimated performance metrics will deviate significantly from the actual quality of results (QoR) achieved by FPGA-targeted designs. The inaccurate estimation prevents designers from evaluating their design

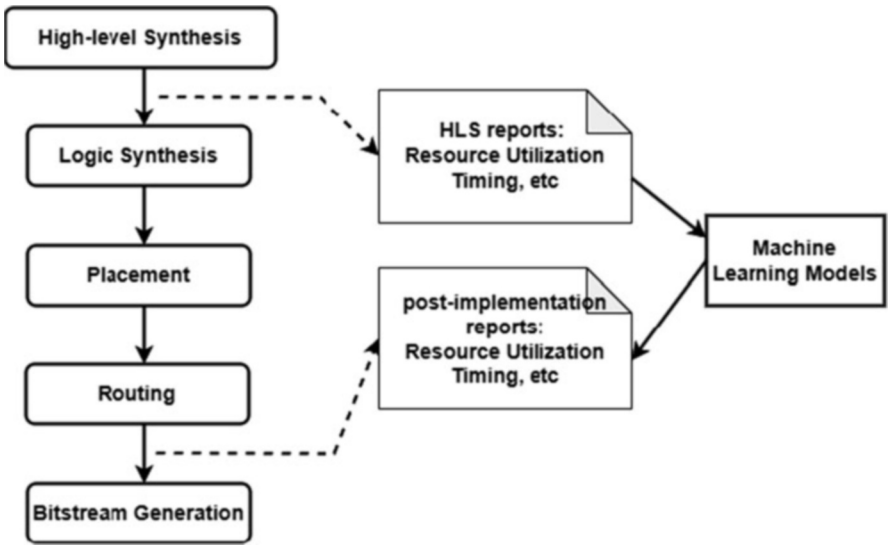


Fig. 7.4 A typical ML-based metric estimation based on FPGA HLS reports

and further exploring the design space. To address this challenge, HLS metric estimation algorithms have been explored with the strength of Artificial Intelligence.

A typical ML-based QoR estimation method is discussed in [25]. Dai et al. built a large collection of C-to-FPGA results from a diverse set of realistic HLS applications and identified relevant features from HLS reports for estimating post-implementation metrics. Then he leveraged these features and data to train and compare a number of promising machine learning models to effectively and efficiently bridge the accuracy gap. As shown in Fig. 7.4, the models are trained by estimates of resource utilization, timing, and HDL details in the HLS reports. The trained models can then be used to predict those metrics for post-implementation. In this work, they focus on regression models, including the linear model, the artificial neural network, and gradient tree boosting. It is provided in the article that their ML models can reduce the errors of the HLS estimations by up to 138%, which proves the effectiveness of AI in the estimation of the HLS metrics.

Similarly, [26] proposed a framework Pyramid based on the ensemble machine learning model for the estimation of the throughput or throughput to area. Features are also extracted from HLS reports, but an automated hardware optimization tool called Minerva is adopted for close-to-optimal results in this work, which employs a customized heuristic algorithm to find the maximum frequency. In this way, they create a comprehensive dataset (2700 samples) with all features from the HLS report, using Pearson's correlation coefficient calculations to remove redundant ones. For ML models, one model from each branch of ML is selected, including the regression model, ANN, support vector machine (SVM), and random forest (RF). In addition, this work proposes an ensemble ML model based on a stacking

Table 7.2 Error of different ML techniques for different benchmarks (Pyramid: machine learning framework to estimate the optimal timing and resource usage of a high-level synthesis design)

Benchmark	Machine learning		Image/video processing		Cryptography		Mathematical	
Targets	Res	Tim	Res	Tim	Res	Tim	Res	Tim
Linear regression	29%	25%	17%	16%	38%	22%	11%	8%
ANN	17%	14%	13%	11%	19%	14%	8%	7%
Support vector machine	22%	19%	18%	17%	23%	18%	10%	7%
Random forest	16%	16%	14%	12%	20%	15%	9%	7%
Ensemble	**6%**	**5%**	**4%**	**3%**	**5%**	**4%**	**4%**	**3%**

approach. Ensemble learning is a segment of machine learning aimed at enhancing the precision and efficacy of standard ML models. This is achieved by creating a collection of base learners and combining their output for the final decision [27, 28]. The performance of the ML models is compared for different benchmarks, with two optimization goals. For all benchmarks, this model based on ensemble learning demonstrates an accuracy of over 95%, as shown in Table 7.2.

Besides the estimation of timing, utilization, and throughput, there are also ML-based estimation methods for power, delay, and routing congestion.

Power consumption is critically important in hardware design, particularly for portable and embedded systems. Recent studies [29–31] have developed power modeling methods to expedite power analysis in hardware design. However, these methods have notable limitations: (1) They produce design-specific power models that are not transferable to other designs, (2) their modeling requires multiple power characterization iterations, following the time-consuming RTL implementation process, and (3) adapting these techniques to new platforms is challenging due to their reliance on specialized hardware modeling knowledge.

To solve this problem, Lin et al. [32] introduce HL-Pow, a power modeling framework for FPGA HLS based on regression models with supervised learning. In this work, an automated feature construction flow is introduced for rapid identification and extraction of features that are most related to power consumption from HLS reports. The HL-Pow model can then offer faster power prediction compared to the time-consuming RTL-based power estimation flow. Lin et al. further propose another framework called PowerGear in [33], which is an improved power estimation method based on graph neural networks (GNNs). The HLS designs are transformed into graph-structured data, where both intraoperation micro-architectures and inter-operation interconnects are encoded and annotated with switching activities. The novel power-aware heterogeneous edge-centric GNN model can effectively learn heterogeneous edge semantics and structural properties of the constructed graphs via edge-centric neighborhood aggregation and fit the formulation of dynamic power. This new ML-based power estimator achieves errors of 3.6 and 8. 81% in estimating the total and dynamic power, along with a speedup of 4× over the Vivado power estimator.

GNNs are a class of neural networks designed to operate directly on graph-structured data, which can perform much better than traditional neural networks in graph-related tasks. Besides the GNN-based power estimation proposed in [33], a GNN-based delay estimation is carried out in [34]. In this work, Ecenur Ustun et al. focus on issues about operation mapping of hardened blocks, such as DSP blocks and carry blocks. Existing HLS tools are found to always fail to capture operation mapping patterns, leading to inaccurate estimation of resource utilization and delay. The proposed GNN model is trained on microbenchmarks directly to realistic designs through inductive learning. The approach leads to a 72% reduction in root mean square error compared to the delay estimation in Vivado HLS.

In addition to traditional metrics that indicate circuit performance, there are also studies on predicting routing congestion. Usually, the routability is difficult to evaluate in the HLS step, which lacks the post-implementation details after Place and Route. This can lead to a much longer development time in those subsequent steps. To solve this problem, Zhao et al. developed an ML-based method to resolve routing congestion in the early design stage [35]. The training dataset is collected by backtracking the congestion metrics of CLBs and linking them with the HLS IR (intermediate representation). And with their trained model, the highly congested parts can be located in the source code, which is a great improvement in developing efficiency.

7.3.3 ML-Based Design Space Exploration in HLS

One distinct advantage of raising the abstraction level to HLS is that it allows the generation of different microarchitectures of equivalent functions without changing the detailed behavioral description. Instead, this is done by setting synthesis options in the form of pragmas or constraints of the number and type of functional units (FUs). To be more specific, designers are able to control the synthesis of arrays (RAM or registers), loops (unroll, partially unroll, no unroll, or pipeline), and functions (inline or not). In this way, designers can generate their circuits with unique areas, performance, and power. The process of achieving Pareto-optimal designs by changing the pragma combinations is called Design Space Exploration (DSE).

One of the main problems of HLS DSE is the large design space to explore, which grows supralinearly with the number of pragma settings. Therefore, the task of ML-based algorithms is to achieve faster and more accurate exploration in the HLS design space.

The state-of-the-art HLS DSE techniques can be basically divided into two categories: synthesis-based approaches and AI-based methods.

As shown in Fig. 7.5, the synthesis-based methods refer to the methods that generate new pragma combinations and let HLS evaluate the effect of these new combinations. This category of DSE methods can be further divided into meta-heuristics, dedicated heuristics, and transfer learning.

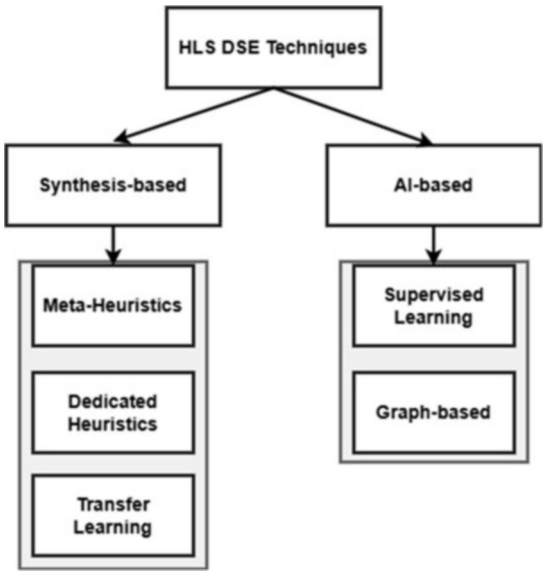


Fig. 7.5 Overview of the HLS DSE techniques

Meta-heuristics are general strategies that are usually based on natural processes, such as simulated annealing (SA), genetic algorithms (GAs), and ant-colony optimizations (ACOs). These techniques have been demonstrated to be highly effective when applied to multi-objective optimization issues like HLS DSE. And they have been proven to be effective in the HLS DSE task [36–40]. However, these methods are limited by their sensitivity to the hyper-parameters. To address this problem, dedicated heuristics are proposed for better performance [41–48]. In addition, transfer learning-based methods are adopted to provide more efficient exploration by making use of previously generated exploration results [49–51]. Instead of testing random pragma combinations, transfer learning can reduce the exploration time with an initiation based on the previous exploration results, which helps to make DSE more efficient and accurate.

AI-based DSE methods are based on predictive models of different performance metrics, which have been discussed in Sect. 7.3.2. Instead of invoking the HLS process every time for pragma combination evaluation, AI-based DSE methods use the generated predictive models for faster evaluation. As shown in Fig. 7.5, AI-based methods can be further classified into supervised learning DSE [52–56] and graph-based DSE [57–59], according to the ML algorithms adopted.

A typical supervised learning-based genetic algorithm for DSE is proposed in [52]. Details of the GA-ML (genetic algorithm machine learning) algorithm are provided in Fig. 7.6. First, the high-level C code is parsed to find all the constructs for exploration. A random set of pragmas for these constructs is generated and then evaluated by the HLS tool. The above step is then repeated to train the predictive ML model. Once the accuracy of the model reaches the given threshold, it can continue the exploration by replacing the HLS tool.

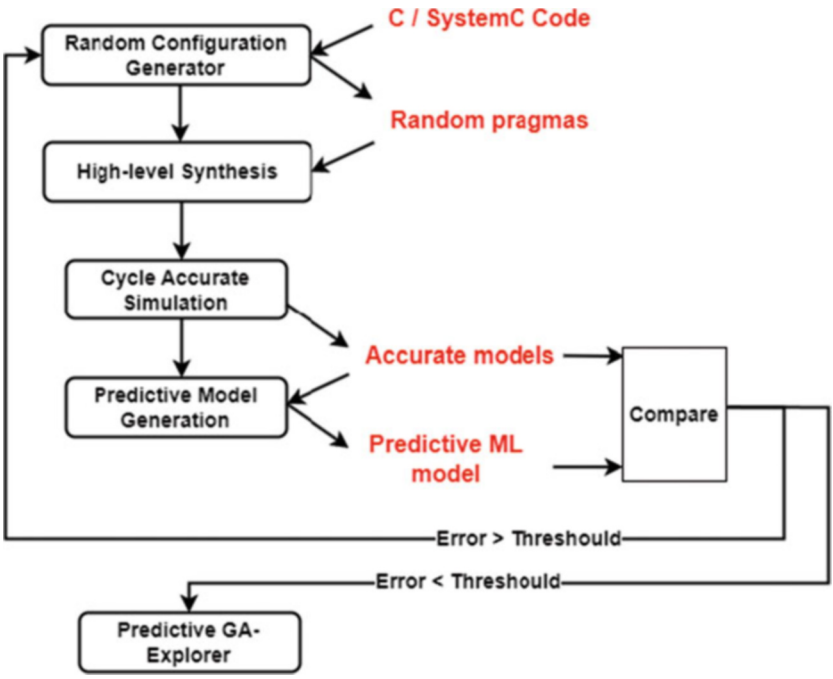


Fig. 7.6 A supervised learning-based algorithm for DSE

Considering that learning-based methods in works like [52] still rely on local search techniques, Liu et al. [53] proposed a better model, especially for HLS, along with an effective training scheme. In their exploration, the Random Forest (RF) is consistently more accurate than the other seven advanced learning models in the prediction of the area and effective latency of a Discrete Fourier Transform (DFT) design. Random Forest (RF) is an ensemble approach that involves numerous decision trees. It works by randomly dividing the training data to train each tree separately. The ultimate prediction is determined by aggregating the predictions from all individual trees through a voting mechanism. The two steps allow RF to minimize both the generalization error and prediction variance and hence achieve high accuracy. In addition to improving the learning model, a transductive experimental design and randomized selection were also included in the proposed method to help find an approximate Pareto set of RTL designs.

Unlike the supervised learning-based DSE methods, graph-based DSE methods generate predictive models and estimate area and performance with features extracted from the Control and Data Flow Graph (CDFG). An example of graph-based DSE methods is the work of Md Imtiaz Rashid in [59]. In this work, a hybrid approach was developed by integrating the conventional metaheuristic technique, Simulated Annealing (SA), with machine learning, resulting in a fast and robust exploration of HLS. Design details are as follows:

Table 7.3 The trade-offs of different HLS DSE methods

Tradeoff (worst=1,best=5)	Meta-Heuristics	Dedicated Heuristics	Transfer learning	Supervised learning	Graph- based
Easy to implement	5	4	3	4	1
Easy to maintain	5	4	3	4	1
Easy to port	5	4	3	/	1
Runtime	2	3	3.5	4	5
Accuracy	5	5	3.5	4	3

(1) The untimed C/C++ description for HLS is first converted into low-level virtual machine (LLVM) intermediate representation (IR) codes and corresponding control and dataflow graphs.
(2) Then a gradient boost-based ML model is trained with the features generated from the LLVM IR and the graphs for the prediction of resources and delays.
(3) The model is then used to build a fast SA design space explorer.

These methods are pretty fast but show a lower accuracy compared to the supervised learning-based methods. This is due to the necessity of fully capturing the behavior of the HLS tool, encompassing all its constraints. Additionally, a drawback is that the models require updates with each revision of the HLS tool, as its behavior could change.

Moreover, it is then found that the area and performance estimation reported after HLS do not always match the results after logic synthesis very well [60, 61]. To build a more accurate ML prediction model, researchers are considering using the results of post-logic synthesis as training data [59]. This has been used by Md Imtaize Rashid to compensate for the low accuracy of their graph-based HLS explorer, which helps to achieve a great improvement in the prediction results.

In conclusion, all methods show different trade-offs between their running time and the accuracy of prediction models. The summary of the trade-offs is provided in Table 7.3.

7.4 AI-Powered Logic Synthesis

Logic synthesis, as suggested by the name, is about transforming the design in HDL format into a netlist of logic gates. Specifically, the RTL code we wrote or generated by HLS in the last step will be converted into the design at the gate level by a synthesis tool. Synthesis tools are vendor-specific computer software that has been integrated into the FPGA design tools, as shown in Table 7.1.

The automation of logic design and synthesis processes has significantly simplified electronic design. By employing high-level programming languages for logic design and synthesis tools for generating logic gates, a level of standardization is achieved among different designers and electronics manufacturers. This not only

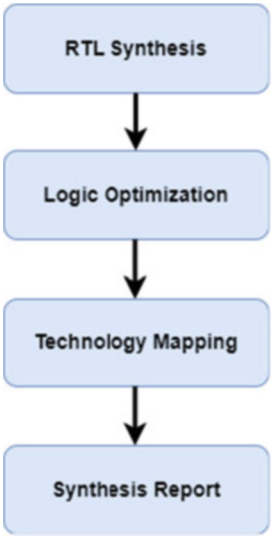


Fig. 7.7 The procedure of logic synthesis

minimizes the likelihood of errors and mistakes but also facilitates the reuse and modification of designs for future applications.

7.4.1 Procedure of FPGA Synthesis

The built-in logic synthesis of an FPGA design tool usually consists of the following four steps: RTL synthesis, logic optimization, technology mapping, and synthesis report generation, which is displayed in Fig. 7.7.

In RTL synthesis, there are three subtasks: syntax analysis, semantic analysis, and modeling. Syntax analysis checks whether the code complies with the syntax rules. Semantic analysis checks the correctness of the code's semantics. Then modeling is about inference and processing of the high-level structures, such as adders, multipliers, RAMs, and FSMs, into gate-level logic. Here, the tool identifies specific patterns within the Verilog logic design or description and processes them according to user-defined instructions.

However, typical logic synthesis involves more than just gate-level translation. It also includes two stages of optimization to improve the quality of the generated circuits. The first optimization is called technology-independent circuit optimization. The circuit is optimized without considering any technology-related metric at this stage. Instead, the optimization objective is to universally enhance the circuit, for instance, by reducing the depth and the quantity of nodes in the Boolean network of the circuit. The conventional approach is to apply a series of optimization steps to the circuit in a predetermined sequence, which has been demonstrated to be effective for typical circuits.

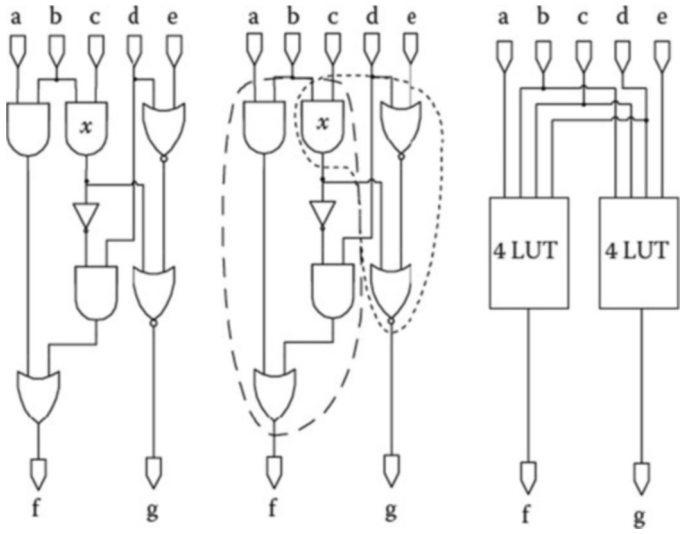


Fig. 7.8 An example of technology mapping in FPGA synthesis (from Hutton et al. "FPGA synthesis and physical design")

The second stage of optimization is technology-dependent optimization, which is also called technology mapping. The network of primitive gates is further mapped into a netlist of primitives that are compatible with the available technology, such as LUTs in FPGA synthesis. LUT-based technology mapping is best seen as a covering problem since it is common and necessary to cover some gates by multiple LUTs for an efficient solution. A more detailed illustration is provided by Ling et al. [62] in Fig. 7.8, which shows how the original netlist is covered and replaced by the four-input LUTs. The optimization goal of this stage is to minimize both the unit depth and the number of cells in the mapped network.

Before synthesis, some setups are also required, such as adding constraint files, specifying the FPGA model, and configuring the synthesis options. Constraint files are essential files in the FPGA implementation process, containing information about time constraints, pin assignments, and clocks. The technology and available resources on different FPGAs can vary greatly, which requires users to provide the FPGA model before synthesis. Synthesis options refer to the settings used by the synthesizer when synthesizing the code, such as optimization levels and timing constraints. Finally, the synthesis tools will generate a synthesis report, which includes information about the synthesis results, resource utilization, timing constraints, etc. The synthesis report contains gate-level implementation details, timing analysis, and constraint verification, allowing you to examine the circuit's implementation and timing performance.

However, conventional logic synthesis methods show restrictions in several aspects. In conventional logic synthesis, a fixed set of optimization algorithms for netlist mapping is applied to different circuits, which is deemed to work well in most

cases. This can now be powered by ML methods to achieve better flexibility in the selection of unique algorithms. Another restriction of conventional logic synthesis is the precision of the estimation of expected performance, resource usage, and timing. The generated HLS reports usually do not show accurate results, which can be significantly improved with ML-based algorithms. Moreover, the problem of slow synthesis time is gaining attention as well. ML is adopted to help deliver timing closure of FPGA synthesis and hence speed up the CAD flow.

7.4.2 Reinforcement Learning-Based Logic Optimization and Technology Mapping

As discussed in the last section, conventional logic synthesis uses only one set of optimization algorithms for different circuits, which shows pretty low flexibility and cannot be auto-adjusted to fit each circuit to its best. In recent research, reinforcement learning (RL) has been explored for improving the two optimization stages in LS.

RL is a state-of-the-art ML paradigm that has shown high performance in sequential decision-making problems. RL model adopts an agent who continually interacts with the environment and is trained by reward and punishment to take correct actions. Considering that the conventional LS approach is made up of a sequence of optimization, RL can be used to help optimize the performance of LS.

In [63], an RL-based logic synthesis is proposed, along with a feature-importance analysis utilizing a random forest technique to trim down the feature space. The overall flow of their proposed framework is displayed in Fig. 7.9, which is a very typical structure for the exploration of LS with RL.

In the framework, they use a popular logic synthesis tool called ABC [64] that uses an AND-inverter graph (AIG) as the subject graph and a feature extractor called CIRC [65] to extract features from the circuit netlist. The RL agent receives the

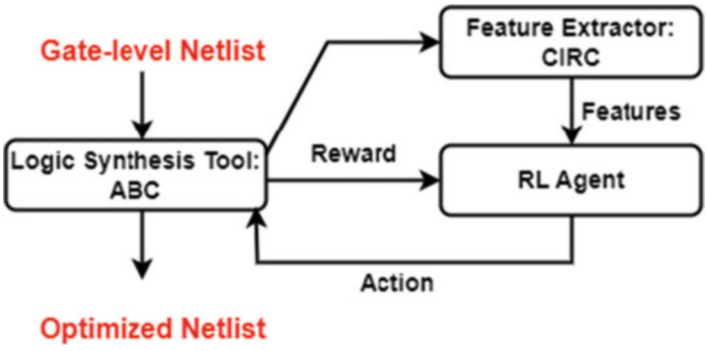


Fig. 7.9 Enhanced logic synthesis with reinforcement learning

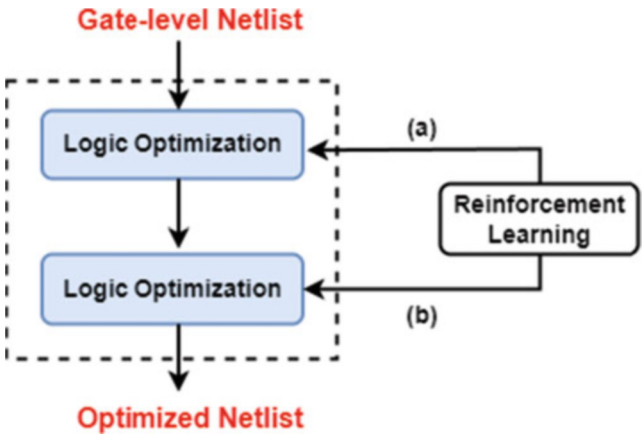


Fig. 7.10 Exploration of logic optimization and technology mapping with RL

observation from both the CIRC tool and the ABC synthesis tool. For every action, the environment provides the agent with observations and rewards. Subsequently, the circuit undergoes technology mapping to attain the area optimization objective. The proposed RL agent proves to finally achieve a 10.5% LUT count improvement and shows its generalization in the optimization of different circuits.

Another RL-based framework called ESE (Efficient Synthesis Exploration) is proposed to automatically explore the logic synthesis process in [66]. In this work, two RL environments are formulated to model the logic optimization and technology mapping, respectively, as Fig. 7.10 shows. The reward functions and termination conditions in RL environments are devised to offer balanced guidance to optimize both metrics and execution time. Specifically, for tackling the FPGA mapping task, logic optimization and technology mapping are merged to enable flexible learning.

7.4.3 Fast and Accurate Synthesis Result Estimation with ML

To solve the problem of inaccurate prediction in the HLS report, ML models are applied to develop fast and accurate estimation predictors for QoR. Some of them have already been discussed in Sect. 7.3.3, where ML models are trained with the features extracted from the HLS reports and the labels from the post-implementation results. For example, with the ML predictor proposed in [25], we can have a better estimation of the final resource utilization and timing directly from the input HLS reports.

Besides high-level synthesis, ML methods are also applied to help designers make early decisions on logic synthesis. At the stage of technology mapping, gate-level netlists will be mapped into circuits with FPGA hardware primitives.

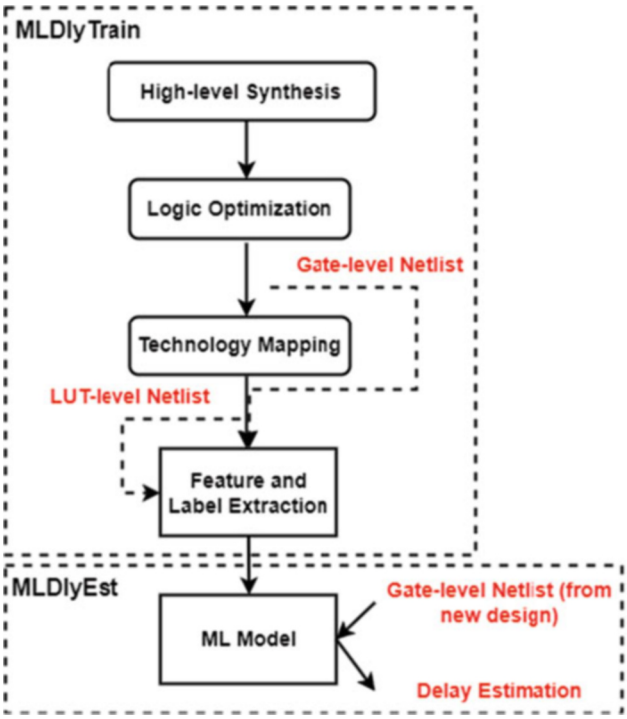


Fig. 7.11 The proposed framework for delay prediction of technology mapping

Considering that early decisions can greatly affect the final timing QoR, it will speed up the synthesis process if we can predict the delay before the time-consuming technology mapping stage. Therefore, in [67], Hu et al. proposed an ML-based delay predictor for quick timing analysis before mapping.

As shown in Fig. 7.11, the proposed framework consists of two parts, MLDlyTrain for training and MLDlyEst for inference. For the training dataset, 12 features are chosen and categorized into four groups, in order to encompass the gate-level netlist's topology and the behavior of technology mapping. These features include the count of Primary Inputs (PI), the count of gates, the count of paths, and the path length. Labels are extracted from the LUT-level netlist after mapping in the format of LUT logic levels. In this work, a supervised learning-based model called Light Gradient Boosting Machine (LGBM) is adopted. LGBM is based on gradient boosting and works well for nonlinear relationships, which makes it a good option for the delay estimation task. The final experiment results also prove its capability with an error of less than 2% for most of the selected benchmarks and an 8× faster runtime.

7.4.4 ML-Powered Time Convergence in FPGA Synthesis

Since the parameter setting in the FPGA CAD tools can affect the final timing results of the circuits as much as 70% without changing the source RTL code, a fast and accurate selection of the parameters is needed. However, the number of parameters and option values is too large in modern CAD tools, with thousands of combinations to select from. The slow runtime of CAD prevents designers from doing a thorough investigation of all combinations.

To solve the problem, InTime, an ML-powered software, is proposed to help FPGA CAD deliver timing closure and has become a plug-in in the FPGA CAD tools of both Xilinx and Altera. The function of InTime is to automatically select and tune the FPGA CAD parameters on behalf of the designers.

As shown in Fig. 7.12, InTime initially performs an analysis and generates a comprehensive set of synthesis and place-and-route configurations using predefined recipes. This approach is similar to the design space exploration tools provided by vendors. However, InTime has a metrics database to save past results and use them to train ML models for strategy updating. With the power of ML, InTime surpasses other DSE tools in performance. Moreover, cloud computing is also adopted for the parallel execution of FPGA CAD. Multiple CAD runs are executed in each round in parallel and can be adjusted with the results from previous rounds. In this way,



Fig. 7.12 Overview of the structure of InTime tool

InTime is capable of reducing the exploration space and achieving a great speedup in the FPGA CAD time convergence.

Although the invention of InTime greatly accelerates the FPGA CAD, including logic synthesis, one key challenge is the accuracy of ML-based prediction on potentially good combinations of CAD parameters. A more accurate prediction makes CAD much easier to achieve time convergence, which reduces not only the development time but also the cloud computing costs for parallel CAD runs.

To further speed up time convergence in FPGA CAD, Yanghua et al. [68] built an accurate estimation flow to help identify parameter combinations that are more likely to improve timing. In this work, a broad set of classification algorithms is explored in terms of their ability to reduce InTime timing scores, including logic regression, bagging, random forest, support vector machine, neural network, stack, and ensemble learning. By configuring a stack-based approach that combines multiple classifiers, they finally achieved a 2.7× reduction in time in timing closure.

7.5 AI-Powered Placement

Placement is part of the physical design flow of FPGA, which consists of packing, placement, and routing. Packing is a unique step for the placement of FPGA, which is usually included in the placement step. In this step, the logic elements are pre-clustered into more compact groups called logic clusters. These logic clusters will be mapped onto specific blocks called CLB, which is the basic logic resource on FPGA, as mentioned in Sect. 7.1. Then in placement, these CLBs will be assigned to physical locations on the FPGA, making this a slot assignment problem, where a CLB must be placed on a location with CLB fabrication.

7.5.1 Conventional Placement Algorithms

In placement, the main goal is to optimize the total wirelength and the timing. For different types of FPGA architecture, different placement algorithms have been developed to achieve the goal.

The Simulated Annealing Algorithm (SA) is a metaheuristic algorithm commonly used to solve combinational optimization problems such as the traveling salesman problem, layout problems, etc. The SA-based placement algorithm is the most widely used FPGA placement algorithm and is suitable for different architectures, especially island-style architectures. The CLBs are kept moving for an optimized location based on a cost function. This method shows great flexibility and can accommodate most objectives, such as the wirelength or timing performance. Random perturbations are applied to continuously optimize the objective function. The CLBs are then required to move until the desired optimization goal is achieved. An example of SA-based placement is shown in Fig. 7.13, which sets the wirelength

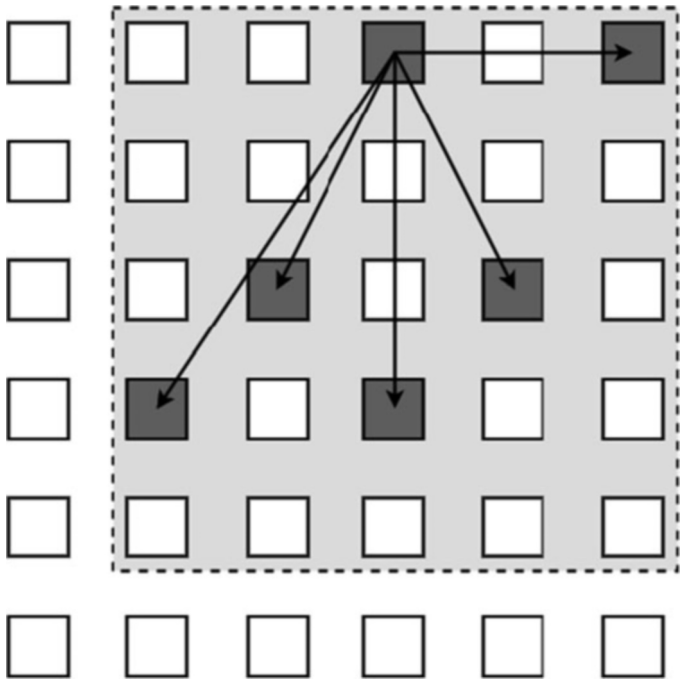


Fig. 7.13 Simulated annealing based placement

as the objective. There are still challenges for SA-based placement. It relies on random perturbations to escape local optima, but there is no guarantee of finding the global optimum. Since the algorithm relies on a sequential exploration of the solution space, the parallelization of it is also challenging. Although there have been improvements in SA, such as multi-CPU acceleration and the VPR algorithm, SA-based placement is still time-consuming compared to the following algorithms.

The partitioning-based placement algorithm is another method for hierarchical architectures. Netlist instances are grouped and used to create clustered hypergraphs based on interconnections. Top-down partitioning is then applied to divide the hypergraphs recursively until all instances are smaller than a predefined threshold. A simple illustration of this process is provided in Fig. 7.14. In this way, partitioning-based placement can transform the original placement problem into mutually orthogonal subproblems, which allows for pretty fast time convergence. However, this method may also easily get trapped in local optima.

The analytical placement algorithm is another popular method in FPGA placement, which models the problem into a continuous optimization problem in ML. The optimal solution to the problem can then be achieved by gradient descent. Based on the objective function, the algorithms can be divided into quadratic placement techniques and nonlinear placement techniques. Compared to the previous two algorithms, analytical placement can deliver great speedup and

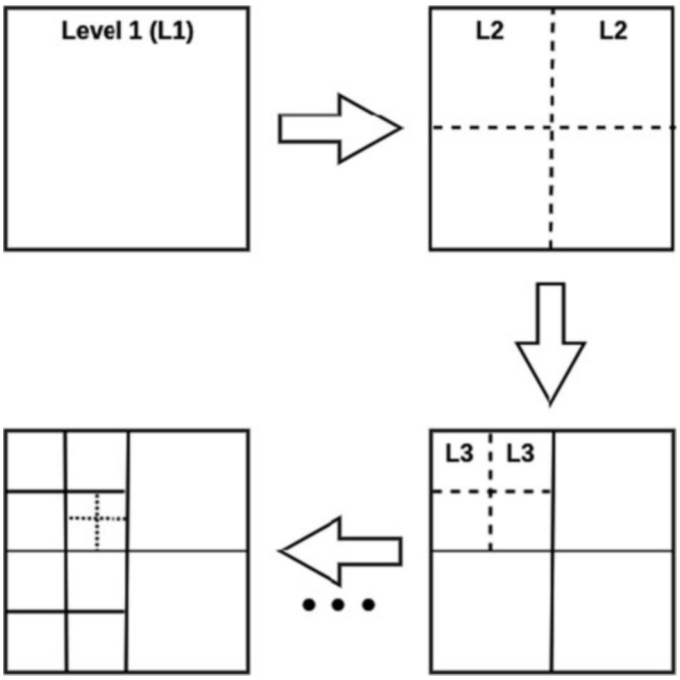


Fig. 7.14 Partitioning-based placement

performance improvement. There are still limitations to this method. The solvers used in Analytical placement require the objective function to be both continuous and differentiable, which does not fit the discrete nature of FPGA placement. Therefore, enhanced analytical placement techniques have been explored to target more complicated FPGAs [69].

However, as one of the most time-consuming steps in CAD, placement has two main tasks, congestion estimation, and routability prediction, in order to prevent potential problems in the routing step. To achieve accurate evaluations, ML techniques are also considered in these tasks.

7.5.2 ML-Based Congestion Estimation

Both placement and routing can consume a large amount of time in the overall CAD flow. A failed routing attempt will lead to iterative placement steps to create congestion-free regions for wiring. Therefore, an accurate post-placement estimation for congestion is necessary for CAD acceleration. Routing congestion is measured as the ratio of the number of routing tracks used to the number of routing tracks available.

Before the AI era, non-ML-based congestion estimation methods have already been explored. Liu et al. proposed a Global Route (GR)-based routing estimation tool for both intercell and intracell nets [70]. In some research, the congestion estimation problem is converted into a graph-based model and solved by routing demand calculation [71]. There are also statistical and probabilistic model-based methods for congestion estimation mentioned in [72].

With technology development and design rule checking (DRC) rules with increasing complexity, ML-powered methods are proposed to learn from previous experience and provide more accurate estimation for congestion.

In [73, 74], the congestion estimation is formed as a binary classification problem. The ML model is trained by features extracted from post-placement or post-routing netlist and will predict whether there will be DRC violations caused by routing congestion. These methods show better performance in modeling the relationship between routing congestion and DRC violations than the conventional global routing-based model. However, there are also limitations, as they do not predict exact congestion inside the global cells.

Supervised learning-based congestion estimation has been explored for an increasing accuracy and lower runtime. In [75], Maarouf et al. improved an SVM-based linear regression model that achieves 95% precision in comparison with the estimates of the Vivado Design Suite, but only 60% in comparison with actual congestion. In this work more features are tested and carefully selected, allowing all regression models in comparison to produce a high R^2 accuracy of more than 85%. The proposed linear regression model for congestion prediction is displayed in Fig. 7.15. This framework is commonly used and widely applicable, serving as the foundation for other machine learning methods [76, 77].

On the basis of the work in [75], Al-Hyari proposed a new congestion estimation framework called MLCong. The training dataset is generated from two academic placers, GPlace3.0 and Pripple. Four features are selected, and all the combinations of them are explored for the accuracy measurements. The researchers also compared different regression models, including linear regression, KNN, ANN, and random decision forests. The final model based on random forest algorithms achieves a 19% reduction in runtime, with a satisfactory level of estimation.

7.5.3 ML-Based Routability Prediction

Traditional non-ML prediction of routability is usually based on the estimated congestion map during placement [77–79]. Then ML-based methods have proven to deliver much better results with the cost of dataset generation and model training.

In [80], a supervised learning-based routability prediction model is proposed. An overview of the ML training procedure is shown in Fig. 7.16. A fast global router is adopted to generate the global routing features, which will be sent to train the ML model along with the features generated from placement results. DRC violations are selected as the label in the training, which is obtained from the post-routing results.

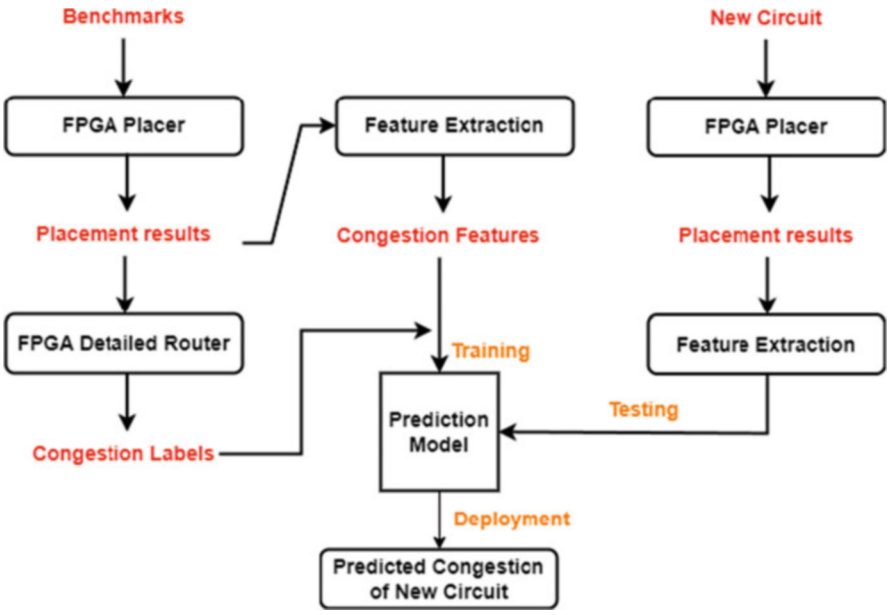


Fig. 7.15 Machine learning-based congestion estimation framework



Fig. 7.16 The training of proposed supervised learning-based routability predictor

With the training dataset prepared, a multivariate adaptive regression is adopted to train the model. Then the trained model can be used to predict the DRC violation before the real detailed routing. The proposed model achieves an average accuracy of 79.8%, and it requires keeping the router and design rules unchanged for a high prediction performance.

Al-Hyari, in the work mentioned above [75], proposed another ML model called MLRoute to predict the routability based on the congestion information generated from MLCong, which shows an accuracy in the range of 93–98%. And later in [81], they upgraded the MLRoute to DLRoute, using CNN to achieve high prediction accuracy within an inference time at the millisecond level.

7.6 AI-Powered Routing

In FPGA routing, the objective is to connect the source blocks to their corresponding destinations using the FPGA's physical resources, ensuring that each routing path is uniquely assigned to a single net from the netlist. And unlike routing for ASIC, FPGA routing is a discrete problem in that the wires have already been implemented in the hardware.

Conventional methods, such as congestion-driven routing algorithm based on negotiation, have been proposed for this task. In these methods, FPGA resources are represented as a directed graph consisting of vertices and edges. Vertices represent the input/output (I/O) channels of different blocks (e.g., logic, DSP, and RAM), while edges indicate potential connections between these blocks. A specific example of such a depiction can be observed in Fig. 7.17.

An example is the PathFinder routing method proposed in [82]. After converting the FPGA routing problem into a directed graph problem, the routing algorithm will route the nets from the source to the destination block iteratively. Initially, there can be resource congestion, where the same resource is shared by multiple nets. By applying a congestion cost function, nets will be made to find a new path with different nodes to reduce the congestion cost. This process is then repeated until a congestion-free routing plan is achieved.

Currently, the two main FPGA vendors, Xilinx and Intel Altera, both use variations of the PathFinder in their products as routers.

7.6.1 ML-Based Routability Prediction

As explained in the previous section, the FPGA routing problem can be transformed into a shortest pathfinding problem in graph theory. Therefore, it can be modeled into different types of problems and solved by diverse ML algorithms.

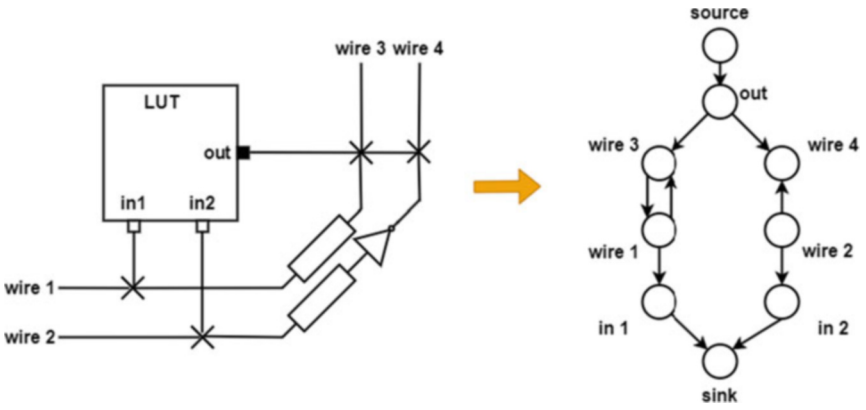


Fig. 7.17 A representation of the FPGA routing resource in the form of directed graph

In [83–86], the routing problem is formulated into continuous decision-making problems. RL-based methods and deep RL (DRL)-based methods are applied to solve it. Haiguang et al. present a DRL-driven global routing method in [83].

In this chapter, a Q-network with a proposed QDN formulation can model the routing resources into a single network and can be solved in a non-heuristics method. The Q-network learns from all previous experience and makes decisions with a reward function. As shown in Fig. 7.18, a problem is adopted to generate problems with specific requirements. Problems are stored separately in text files and then parsed. Each net will be decomposed into a set of two-pin problems using a minimum spanning tree algorithm. Two routers, the A* router and the proposed DQN router, are employed to resolve these two-pin problems. Here, the A* router will provide the memory burn-in to the DQN. The final performance for each router is generated by merging solutions for individual nets and will be evaluated and compared in the format of total congestion. Here, an overview of the DQN router is also provided in Fig. 7.19, showing how it interacts with the environment.

Similarly, Farooq et al. chose to use RL techniques to speed up the routing process [84]. In this work, they adopt a move-based RL model, which changes the reward at each move, making it a dynamic RL problem. The results demonstrate that this method can achieve a 30% reduction in runtime. In [87], however, Jain models the routing as a binary segmentation problem and proposed a Convolutional Neural

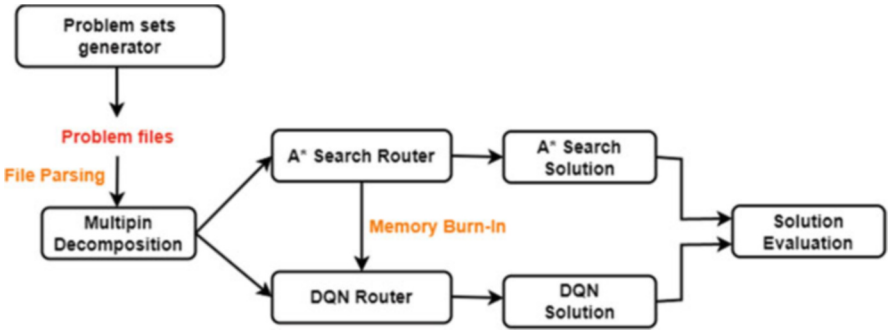


Fig. 7.18 Deep Q-network-based approach for global routing

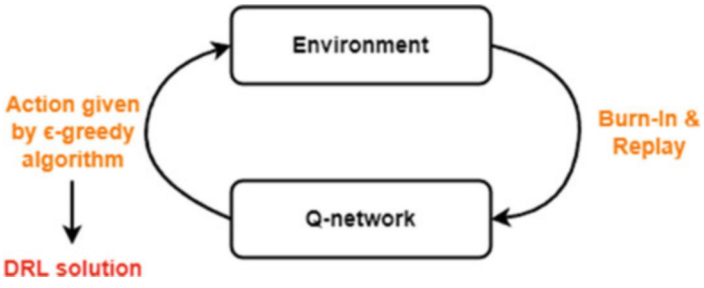


Fig. 7.19 DQN-based router

Network (CNN)-based router. The model is trained to learn the implicit design rules. In this work, they created their own datasets in the format of layout images with eight layers (for pins, metal layers, and vias). Each layer is binary encoded at the pixel level. The pixels in each layout layer are classified by the CNN router as on or off, which is used for the routing decision.

References

1. Tarafdar, N., et al. (2017). Enabling flexible network FPGA clusters in a heterogeneous cloud data center. In *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*.
2. Weerasinghe, J., et al. (2016). Network-attached FPGAs for data center applications. In *2016 International Conference on Field-Programmable Technology (FPT)*. IEEE.
3. Guillot, A., et al. (2017). Multi-FPGA platform for smart automobiles embedded electronics design and prototyping. In *Proceedings of the International Conference on Embedded Systems, Cyber-physical Systems, and Applications (ESCS). The Steering Committee of The World Congress in Computer Science, Computer Engineering and Applied Computing (WorldComp)*.
4. Bhavani, M., & Devi, D. A. (2019). Design of smart Monitor for automobiles using FPGA based data logger. In *2019 International Conference on Communication and Electronics Systems (ICCES)*. IEEE.
5. Liu, L., et al. (2012). Downlink MIMO in LTE-advanced: SU-MIMO vs. MU-MIMO. *IEEE Communications Magazine, 50*(2), 140–147.
6. Liu, L., Zhang, J., & Pi, Z. (2012). Inter-cell interference avoidance for downlink transmission. *U.S. Patent No. 8,238,954*.
7. Shafin, R., et al (2020). Artificial intelligence-enabled cellular networks: A critical path to beyond-5G and 6G. *IEEE Wireless Communications, 27*(2), 212–217.
8. Chang, H.-H., et al. (2018). Distributive dynamic spectrum access through deep reinforcement learning: A reservoir computing-based approach. *IEEE Internet of Things Journal, 6*(2), 1938–1948.
9. Kastensmidt, F., Rech, P. (2016). *FPGAs and parallel architectures for aerospace applications. Soft errors and fault-tolerant design*.
10. Lattuada, M., Ferrandi, F., & Perrotin, M. (2016). Computer assisted design and integration of FPGA accelerators in aerospace systems. In *2016 IEEE Aerospace Conference*. IEEE.
11. Curzel, S., et al. (2023). Exploration of synthesis methods from simulink models to FPGA for aerospace applications. In *Proceedings of the 20th ACM International Conference on Computing Frontiers*.
12. Hosking, R. (2006) New FPGAs tackle real-time DSP tasks for defense applications. In *Boards & Solutions Magazine (November 2006)*.
13. Soumya, D. S., Aishwarya, C., & Vasavi, S. (2023). FPGA-based military vehicles detection and classification from drone videos using YOLOV5. In *Energy Systems, Drives and Automations: Proceedings of ESDA 2021* (Vol. 1057, p. 265).
14. Pillutla, N., & Kumar, S. (2023). FPGA implementation of multi-RT protocol for avionics simulation. In *2023 IEEE International Systems Conference (SysCon)*. IEEE.
15. Lin, C., Liang, Y., & Yi, Y. (2022). FPGA-based reservoir computing with optimized reservoir node architecture. In *2022 23rd International Symposium on Quality Electronic Design (ISQED)*. IEEE.
16. Gan, V. M., et al. (2021). A cost-efficient digital ESN architecture on FPGA for OFDM symbol detection. *ACM Journal on Emerging Technologies in Computing Systems (JETC), 17*(4), 1–15.

17. Shears, O., et al. (2021). A hybrid FPGA-ASIC delayed feedback reservoir system to enable spectrum sensing/sharing for low power IoT devices ICCAD special session paper. In *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE.
18. Lin, C., Azmine, M. F., & Yi, Y. (2023). Accelerating next-G wireless communications with FPGA-based AI accelerators. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE.
19. Yi, Y., et al. (2016). FPGA based spike-time dependent encoder and reservoir design in neuromorphic computing processors. *Microprocessors and Microsystems, 46*, 175–183.
20. Platforms, X. V. (2022). Available online: https://www.xilinx.com/products/design-tools/vitis/vitis-platform.html (Accessed on December 9, 2022).
21. EC2, A. (2022). Available online: https://aws.amazon.com/ec2/instance-types/f1/ (Accessed on December 9, 2022).
22. Nane, R., et al. (2015). A survey and evaluation of FPGA high-level synthesis tools. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 35*(10), 1591–1604.
23. Xilinx (2016). *Vivado High-level Synthesis UG902*. http://www.xilinx.com/
24. Intel (2016). Intel FPGA SDK for OpenCL. http://www.altera.com/.
25. Dai, S., et al. (2018). Fast and accurate estimation of quality of results in high-level synthesis with machine learning. In *2018 IEEE 26th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE.
26. Makrani, H. M., et al. (2019). Pyramid: Machine learning framework to estimate the optimal timing and resource usage of a high-level synthesis design. In *2019 29th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE.
27. Sayadi, H., et al. (2018). Ensemble learning for effective run-time hardware-based malware detection: A comprehensive analysis and classification. In *Proceedings of the 55th Annual Design Automation Conference*.
28. Sayadi, H., et al. (2019). 2smart: A two-stage machine learning-based approach for run-time specialized hardware-assisted malware detection. In *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE.
29. Lee, D., et al. (2015). Learning-based power modeling of system-level black-box IPs. In *2015 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE.
30. Lin, Z., Sinha, S., & Zhang, W. (2018). An ensemble learning approach for in-situ monitoring of FPGA dynamic power. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 38*(9), 1661–1674.
31. Liang, Y., Wang, S., & Zhang, W. (2018). FlexCL: A model of performance and power for OpenCL workloads on FPGAs. *IEEE Transactions on Computers, 67*(12), 1750–1764.
32. Lin, Z., et al. (2020). HL-Pow: A learning-based power modeling framework for high-level synthesis. In *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE.
33. Lin, Z., et al. (2022). PowerGear: Early-stage power estimation in FPGA HLS via heterogeneous edge-centric GNNs. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE.
34. Ustun, E., et al. (2020). Accurate operation delay prediction for FPGA HLS using graph neural networks. In *Proceedings of the 39th International Conference on Computer-Aided Design*.
35. Zhao, J., et al. (2019). Machine learning based routing congestion prediction in FPGA high-level synthesis. In *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE.
36. Schafer, B. C., Takenaka, T., & Wakabayashi, K. (2009). Adaptive simulated annealer for high level synthesis design space exploration. In *2009 International Symposium on VLSI Design, Automation and Test*. IEEE.
37. Ferrandi, F., et al. (2008). A multi-objective genetic algorithm for design space exploration in high-level synthesis. In *2008 IEEE Computer Society Annual Symposium on VLSI*. IEEE.

38. Wang, Z., & Schafer, B. C. (2020). Machine learning to set meta-heuristic specific parameters for high-level synthesis design space exploration. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE.
39. Wang, G., et al. (2006). Design space exploration using time and resource duality with the ant colony optimization. In *Proceedings of the 43rd annual Design Automation Conference*.
40. Mishra, V. K., & Sengupta, A. (2014). PSDSE: Particle swarm driven design space exploration of architecture and unrolling factors for nested loops in high level synthesis. In *2014 Fifth International Symposium on Electronic System Design*. IEEE.
41. Schafer, B. C., & Wakabayashi, K. (2009). Design space exploration acceleration through operation clustering. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 29*(1), 153–157.
42. Xydis, S., et al. (2010). Efficient high level synthesis exploration methodology combining exhaustive and gradient-based pruned searching. In *2010 IEEE Computer Society Annual Symposium on VLSI*. IEEE.
43. Pham, N. K., et al. (2015). Exploiting loop-array dependencies to accelerate the design space exploration with high level synthesis. In *2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE.
44. Cilardo, A., & Gallo, L. (2015). Interplay of loop unrolling and multidimensional memory partitioning in HLS. In *2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE.
45. Ferretti, L., Ansaloni, G., & Pozzi, L. (2018). Cluster-based heuristic for high level synthesis design space exploration. *IEEE Transactions on Emerging Topics in Computing, 9*(1), 35–43.
46. Ferretti, L., Ansaloni, G., & Pozzi, L. (2018). Lattice-traversing design space exploration for high level synthesis. In *2018 IEEE 36th International Conference on Computer Design (ICCD)* (pp. 509–512). IEEE.
47. Schafer, B. C. (2015). Probabilistic multiknob high-level synthesis design space exploration acceleration. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 35*(3), 394–406.
48. Xydis, S., et al. (2014). SPIRIT: Spectral-aware Pareto iterative refinement optimization for supervised high-level synthesis. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 34*(1), 155–159.
49. Wang, Z., Chen, J., & Schafer, B. C. (2020). Efficient and robust high-level synthesis design space exploration through offline micro-kernels pre-characterization. In *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE.
50. Ferretti, L., et al. Leveraging prior knowledge for effective design-space exploration in high-level synthesis. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 39*(11), 3736–3747.
51. Wang, Z., & Schafer, B. C. (2022). Learning from the past: Efficient high-level synthesis design space exploration for FPGAs. *ACM Transactions on Design Automation of Electronic Systems (TODAES), 27*(4), 1–23.
52. Schafer, B. C., & Wakabayashi, K. (2012). Machine learning predictive modelling high-level synthesis design space exploration. *IET Computers & Digital Techniques, 6*(3), 153–159.
53. Liu, H.-Y., & Carloni, L. P. (2013). On learning-based methods for design-space exploration with high-level synthesis. In *Proceedings of the 50th Annual Design Automation Conference*.
54. Zhong, G., et al. (2017). Design space exploration of FPGA-based accelerators with multi-level parallelism. In *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017*. IEEE.
55. Meng, P., et al. (2016). Adaptive threshold non-pareto elimination: Re-thinking machine learning for system level design space exploration on FPGAs. In *2016 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE.
56. Mahapatra, A., & Schafer, B. C. (2014). Machine-learning based simulated annealer method for high level synthesis design space exploration. In *Proceedings of the 2014 Electronic System Level Synthesis Conference (ESLsyn)*. IEEE.

57. Zhong, G., et al. (2016). Lin-analyzer: A high-level performance analysis tool for FPGA-based accelerators. In *Proceedings of the 53rd Annual Design Automation Conference*.
58. Zhao, J., et al. (2017). COMBA: A comprehensive model-based analysis framework for high level synthesis of real applications. In *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE.
59. Goswami, P., Schaefer, B. C., & Bhatia, D. (2023). Machine learning based fast and accurate high level synthesis design space exploration: From graph to synthesis. *Integration, 88*, 116–124.
60. Liu, D., & Schafer, B. C. (2016). Efficient and reliable high-level synthesis design space explorer for FPGAs. In *2016 26th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE.
61. Makrani, H. M., et al. (2019). Pyramid: Machine learning framework to estimate the optimal timing and resource usage of a high-level synthesis design. In *2019 29th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE.
62. Ling, A., Singh, D. P., & Brown, S. D. (2005). FPGA technology mapping: A study of optimality. In *Proceedings of the 42nd Annual Design Automation Conference*.
63. Zhou, G., & Anderson, J. H. (2023). Area-driven FPGA logic synthesis using reinforcement learning. In *Proceedings of the 28th Asia and South Pacific Design Automation Conference*.
64. Mishchenko, A. (2007). *ABC: A system for sequential synthesis and verification*. http://www.eecs.berkeley.edu/alanmi/abc.
65. Hutton, M. D., et al. (1998). Characterization and parameterized generation of synthetic combinational benchmark circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 17*(10), 985–996.
66. Qian, Y., et al. (2022). Efficient reinforcement learning framework for automated logic synthesis exploration. In *2022 International Conference on Field-Programmable Technology (ICFPT)*. IEEE.
67. Hu, H., et al. (2022). Machine-learning based delay prediction for FPGA technology mapping. In *Proceedings of the 24th ACM/IEEE Workshop on System Level Interconnect Pathfinding*.
68. Yanghua, Q., et al. (2016). Improving classification accuracy of a machine learning approach for fpga timing closure. In *2016 IEEE 24th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE.
69. Gort, M., & Anderson, J. H. (2012). Analytical placement for heterogeneous FPGAs. In *22nd International Conference on Field Programmable Logic and Applications (FPL)*. IEEE.
70. Shojaei, H., Davoodi, A., & Linderoth, J. T. (2011). Congestion analysis for global routing via integer programming. In *2011 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE.
71. Kannan, P., Balachandran, S., & Bhatia, D. (2001). fGREP-fast generic routing demand estimation for placed FPGA circuits. In *Field-Programmable Logic and Applications: 11th International Conference, FPL 2001 Belfast, Northern Ireland, August 27–29, 2001 Proceedings 11*. Springer.
72. Saeedi, M., Zamani, M. S., & Jahanian, A. (2007). Evaluation, prediction and reduction of routing congestion. *Microelectronics Journal, 38*(8–9), 942–958.
73. Chan, W.-T. J., et al. (2017). Routability optimization for industrial designs at sub-14nm process nodes using machine learning. In *Proceedings of the 2017 ACM on International Symposium on Physical Design*.
74. Tabrizi, Fakheri, A., et al. A machine learning framework to identify detailed routing short violations from a placed netlist. In *Proceedings of the 55th Annual Design Automation Conference*.
75. Maarouf, D., et al. (2018). Machine-learning based congestion estimation for modern FPGAs. In *2018 28th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE.
76. Al-Hyari, A., et al. (2019). Novel congestion-estimation and routability-prediction methods based on machine learning for modern FPGAs. *ACM Transactions on Reconfigurable Technology and Systems (TRETS), 12*(3), 1–25.

77. Maarouff, D., et al. (2020). A deep-learning framework for predicting congestion during FPGA placement. In *2020 30th International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE.
78. Yu, C., Zhang, Z. (2019). Painting on placement: Forecasting routing congestion using conditional generative adversarial nets. In *Proceedings of the 56th Annual Design Automation Conference 2019*.
79. Alawieh, M. B., et al. (2020). High-definition routing congestion prediction for large-scale FPGAs. In *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE.
80. Zhou, Q., et al. (2015). An accurate detailed routing routability prediction model in placement. In *2015 6th Asia Symposium on Quality Electronic Design (ASQED)*. IEEE.
81. Al-Hyari, A., et al. (2021). A deep learning framework to predict routability for FPGA circuit placement. *ACM Transactions on Reconfigurable Technology and Systems (TRETS), 14*(3), 1–28.
82. McMurchie, L., & Ebeling, C. (1995). PathFinder: A negotiation-based performance-driven router for FPGAs. In *Proceedings of the 1995 ACM Third International Symposium on Field-Programmable Gate Arrays*.
83. Liao, H., et al. (2020). A deep reinforcement learning approach for global routing. *Journal of Mechanical Design, 142*(6), 061701.
84. Farooq, U., et al. (2021). Efficient FPGA routing using reinforcement learning. In *2021 12th International Conference on Information and Communication Systems (ICICS)*. IEEE.
85. He, Y., et al. (2022). Circuit routing using Monte Carlo tree search and deep reinforcement learning. In *2022 International Symposium on VLSI Design, Automation and Test (VLSI-DAT)*. IEEE.
86. Baig, I., & Farooq, U. (2022). Efficient detailed routing for FPGA back-end flow using reinforcement learning. *Electronics, 11*(14), 2240.
87. Jain, S. R., & Okabe, K. (2017). Training a fully convolutional neural network to route integrated Circuits. *arXiv preprint arXiv:1706.08948*.